What Testing Practices Do Blockchain Agent Skills Instruct?
A Testing Skill Profile of GitSkills

Anonymous Author(s)

***Abstract*—[TODO: abstract (deadline 18 Dec 2026)]**

***Index Terms*—Agent Skills, Software Testing, Test Pyramid, Blockchain-Oriented Software Engineering, Mining Software Repositories**

I. Introduction

[TODO: motivation: agent skills instruct coding agents in natural language, with no compiler checking their testing guidance; in blockchain, faults in value-bearing code are costly and irreversible.]

We investigate which testing practices the agent skills of the blockchain domain instruct the agent to follow, using the Practical Test Pyramid [1] as reference. We address two research questions:

RQ1 (Profile). Which test layers do the skills instruct, and in which context (on-chain, off-chain Web3, Web2)?

RQ2 (Practices). Which good and bad testing practices do the skills instruct, and how often per category and context?

[TODO: contributions C1–C3: Testing Skill Profile; catalog of practices with quotes; reusable instrument and annotations.]

II. Methodology

Our methodology follows a Mining Software Repositories (MSR) approach structured in six phases, as shown in Figure 1. The unit of analysis is one distinct `SKILL.md` file. All scripts use a fixed seed (20270402), every execution is logged, and every number reported in this paper is read from a file generated by a script, available in the replication package [2].

A. Phase 1: Skill Selection

Related to the **Data Source**, we use GitSkills [3], a snapshot of the `SKILL.md` files publicly visible on GitHub in July 2026, with 1,877,981 distinct contents from 282,200 repositories. GitSkills groups verbatim copies by content hash and marks one representative per group (`dedup_primary` $= 1$), which we use to avoid counting the same text more than once. We start from the subset of 89,649 distinct skills that match at least one term of a broad blockchain vocabulary. [TODO: re-run Phase 1 on the full 41 GB database and update this sentence.]

In **Domain Scoping**, we restrict this subset to the blockchain domain with a vocabulary split into two classes. *Strong* terms are sufficient on their own (e.g., “solidity”, “smart contract”, “hardhat”, “forge test”), while *weak* terms are ambiguous outside the domain and only count when two distinct weak terms co-occur or when a strong term is present (e.g., “wallet”, “gas”, “token address”). The terms were proposed by an LLM and reviewed by two researchers, resulting in 75 strong and 17 weak terms. We match them as case-insensitive patterns delimited by non-alphanumeric characters, applied to the name, description, and content of each skill. We adopted this split because a first version, in which weak terms were generic words in LLM-oriented skills (e.g., “tokens”, “layer 1”, “validators”), admitted mostly Web2 skills such as planning and review workflows. Unambiguous forms were kept as strong (e.g., “foundry.toml” rather than “foundry”, which also names cloud AI platforms), and generic forms were removed or qualified. The domain scope comprises 28,244 skills, of which 27,736 contain a strong term and 508 entered through weak terms only.

In **Test Filtering**, we keep the skills that contain at least one of 56 compound testing phrases, generated and reviewed in the same way (e.g., “unit test”, “forge test”, “fuzz”, “invariant test”, “playwright”). We never use the isolated word “test”, which occurs in many skills that do not address testing. This filter only selects candidates; the classification itself is carried out in Phases 4 and 5. The resulting pool has 4,928 skills from 2,100 repositories. Since 1,073 of these contents have verbatim copies (up to 302 each), the pool corresponds to 12,314 file occurrences in GitSkills.

In **Selection Validation**, one researcher checks 100 skills randomly drawn from the domain scope (“is this skill about blockchain/Web3?”) and 100 from the pool (“does this skill address testing?”), blind to the matched terms. We report the precision of each step with a 95% Wilson confidence interval [4], with a target of at least 0.90. Precision was [TODO: domain: p [CI]; test: p [CI]].

B. Phase 2: Instrument

The same text serves as the human guideline and as the LLM prompt, so that both apply identical criteria. Each item is answered with a single question: *does the text instruct the agent to do this?* Mentioning or listing a practice does not count.

Context and domain class. Each piece of evidence receives a context defined by what the tested component does, not by the repository where it lives: *on-chain* (code executed by the blockchain, e.g., Solidity, Vyper or Anchor programs, proxies, deploy scripts), *off-chain Web3* (code outside the chain that reads or writes chain state, signs transactions, manages keys, or consumes ABIs and events, e.g., dApps with a wallet,

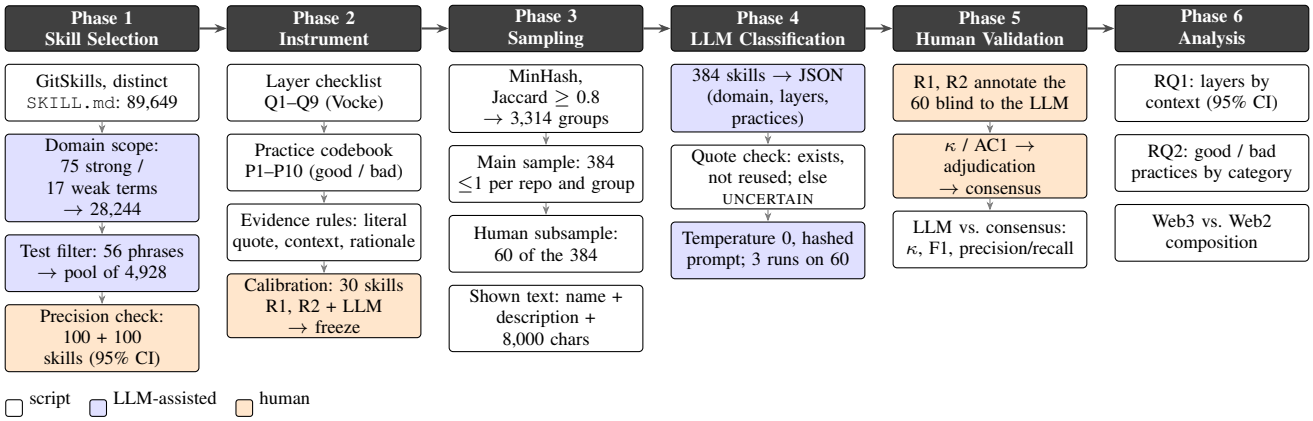


Fig. 1. Methodology in six phases. Numbers are those of the current execution (seed 20270402).

SDK/RPC clients, indexers, bots, bridges), or *Web2* (code with no chain interaction, even inside a Web3 project). Each skill also receives a domain class: `onchain`, `offchain_web3`, `mixed`, `web2_incidental` (blockchain is a peripheral mention), or `undetermined`. This class answers whether the skills in our scope are predominantly Web3 or Web2.

Layer checklist. Nine items cover the layers of the Practical Test Pyramid [1]: Q1 Unit, Q2 Integration, Q3 Contract, Q4 UI, Q5 End-to-End, Q6 Acceptance, Q7 Exploratory, and Q8 Automation, plus Q9 Web3-specific techniques (fork, fuzz, invariant, formal verification, and gas testing). The YES criterion of each item paraphrases the corresponding section of Vocke [1], and its adaptation to blockchain follows the Foundry, Hardhat, and Anchor documentation [5]–[7]. For example, Q1 is YES when the skill instructs testing a contract function with mocked dependencies, or an SDK function with a stubbed RPC; "run the tests" alone does not count. Similarly, Q2 requires a real junction (contract to contract on a local chain, or backend to RPC node), and a mainnet fork without checking the junction does not count.

Practice codebook. Ten categories capture good and bad testing practices: P1 pyramid balance, P2 isolation, P3 assertions, P4 determinism, P5 adversarial testing, P6 automation and CI, P7 coverage, P8 test integrity, P9 secrets, and P10 other practices emerging in calibration. Polarity is anchored in sources rather than in the annotator's opinion: Vocke [1], the test smells catalog of Garousi and Küçük [8], the Foundry Book [5], the ConsenSys Smart Contract Best Practices [9], and the OWASP Smart Contract Top 10 [10]. For instance, pinning a fork to a block number is a good P4 practice, whereas a fork without a pinned block is a bad one; fixing the code is a good P8 practice, whereas skipping or loosening a failing test is a bad one.

Evidence rules. Each YES and each practice requires a literal quote of 5 to 40 words and a one-sentence justification linking the quote to the criterion or category, together with its context. Only what is instructed counts, and a quote cannot justify two different items. In **Calibration**, two researchers and the LLM annotate 30 skills outside the sample, adjust the examples once (including new P10 categories), and freeze the instrument. [TODO: calibration outcome: changes and new P10 categories.]

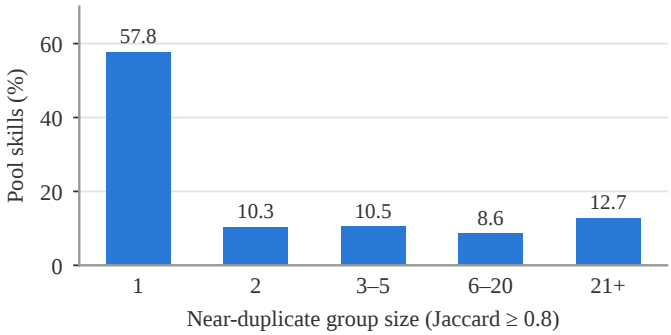


Fig. 2. Share of the 4,928 pool skills by the size of their near-duplicate group. 57.8% are unique, while 12.7% belong to templates with more than 20 variants.

C. Phase 3: Sampling

GitSkills reports that 50.5% of all file occurrences are verbatim copies [3], and templates are also copied with small edits. To prevent a single template from dominating the sample, we group **near-duplicates** with MinHash [11] (128 permutations over 5-word shingles, locality-sensitive hashing with 16 bands) and link two skills when their estimated Jaccard similarity is at least 0.8. The pool forms 3,314 groups, and 2,078 skills (42.2%) belong to a group with two or more members; the largest group has 151 machine-generated skills sharing the same boilerplate. Figure 2 shows that only 57.8% of the pool skills are unique, and 12.7% belong to templates with more than 20 variants, which would otherwise dominate any random sample.

For the **Main Sample**, we draw 384 skills from the pool, which gives a 95% confidence level with a $\pm 5\%$ margin [12]. We shuffle the pool with the fixed seed and accept a skill only if neither its repository nor its near-duplicate group has been selected, so the sample holds at most one skill per repository and per group. The 30 calibration skills are drawn next under

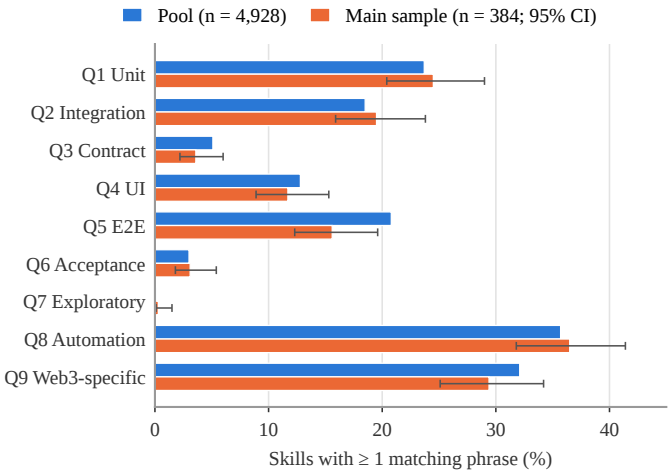


Fig. 3. Lexical testing signals per checklist layer: share of skills whose text contains at least one test phrase mapped to the layer. These are term matches used for candidate selection, not the classification of Phases 4–5 (e.g., "contract testing" also matches "smart contract testing").

the same constraints, so they share no repository or group with the main sample. A **Human Subsample** of 60 skills is drawn within the 384. The text shown to the LLM and to the annotators is identical: name, description, and content up to 8,000 characters; 246 skills (64.1%) of the main sample exceed this limit and are truncated.

Figure 3 compares the pool and the main sample on lexical testing signals, i.e., the test phrases of Phase 1 mapped to the checklist layers. The pool value falls within the 95% confidence interval of the sample for eight of the nine layers. The exception is E2E (20.8% in the pool versus 15.6% in the sample), because a single end-to-end testing template copied across dozens of repositories inflates the pool and is represented only once in the sample. These signals only describe the candidates; whether the skills actually *instruct* each layer is decided in Phases 4 and 5.

D. Phase 4: LLM Classification

The LLM[1] classifies each of the 384 skills and returns a JSON object with the domain class; the answer, quote, justification, and context for Q1–Q9; and a list of practices, each with quote, category, polarity, context, and justification. An **Automatic Check** verifies that every quote exists in the shown text and is not reused across items; an item that fails twice becomes UNCERTAIN and is counted as discarded. We use temperature 0, record the SHA-256 hash of the prompt, and run the 60 skills of the subsample three times to measure the stability of the answers. [TODO: discard rate; stability across the three runs.]

E. Phase 5: Human Annotation and Validation

In **Blind Annotation**, two researchers (R1 and R2) independently annotate the 60 skills of the subsample without seeing the LLM output, using per-annotator sheets in which skills are identified only by an opaque ID. We measure **Human Agreement** with Cohen's κ [13] per item, and Gwet's AC1 [14] for rare items, where κ is unstable; disagreements are then adjudicated into a consensus. To compare the **LLM with the Consensus**, we compute κ and F1 per layer, per context, and per domain class. For practices, we compute precision and recall, matching quotes that overlap by at least 50%, and κ for category and polarity. Any item in which the LLM reaches $\kappa < 0.40$ is reported with the human values only. [TODO: agreement values (Tab. II).]

F. Phase 6: Analysis

For RQ1, we report the percentage of skills with YES per layer, with 95% confidence intervals, split by context, and the Web3 versus Web2 composition of the scope. For RQ2, we count good and bad practices per category and context, with a short example of each polarity.

III. Results

[TODO: RQ1: Fig. 2 (profile by context). RQ2: Tab. I (catalog of practices).]

IV. Discussion

[TODO:]

V. Threats to Validity

[TODO: term-based selection (precision measured in Phase 1); subset of GitSkills rather than the full snapshot; LLM over-classification (consensus as reference, mandatory quotes, automatic check); subjective polarity (anchored in sources, agreement measured); on/off-chain/Web2 boundary (explicit decision rule, κ reported); truncation of 64.1% of the sample at 8,000 characters; instruction is not actual practice; copies (deduplication and one skill per repository and group).]

VI. Ethics

[TODO: mandatory section: public data, no personal data, anonymised authors in GitSkills, annotation by the authors only.]

VII. Conclusion and Data Availability

[TODO:] The checklist, codebook, prompt, scripts, and annotations are available in an anonymized repository [2].

[1] [TODO: exact model and version.]

References

[1] H. Vocke, "The practical test pyramid," https://martinfowler.com/articles/practical-test-pyramid.html, 2018.

[2] "Replication package (anonymized)," [TODO: anonymized repository URL].

[3] G. Destefanis, D. Graziotin, M. Vaccargiu, and M. Ortu, "GitSkills: A dataset of agent skills on GitHub," in *Proceedings of the 24th International Conference on Mining Software Repositories (MSR)*, 2027, mining Challenge. Preprint arXiv:2608.10906. Data: doi:10.5281/zenodo.21875637.

[4] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[5] "The Foundry book," https://book.getfoundry.sh, accessed: 2026-10.

[6] "Hardhat documentation: Testing contracts," https://hardhat.org/docs, accessed: 2026-10.

[7] "Anchor documentation," https://www.anchor-lang.com/docs, accessed: 2026-10.
[8] V. Garousi and B. Küçük, "Smells in software test code: A survey of knowledge in industry and academia," *Journal of Systems and Software*, vol. 138, pp. 52–81, 2018.
[9] ConsenSys Diligence, "Ethereum smart contract security best practices," https://consensys.github.io/smart-contract-best-practices/, accessed: 2026-10.
[10] OWASP, "OWASP smart contract top 10," https://owasp.org/www-project-smart-contract-top-10/, accessed: 2026-10.
[11] A. Z. Broder, "On the resemblance and containment of documents," in *Proceedings of Compression and Complexity of Sequences*, 1997, pp. 21–29.
[12] W. G. Cochran, *Sampling Techniques*, 3rd ed. Wiley, 1977.
[13] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
[14] K. L. Gwet, "Computing inter-rater reliability and its variance in the presence of high agreement," *British Journal of Mathematical and Statistical Psychology*, vol. 61, no. 1, pp. 29–48, 2008.